

7.2 Mixture of Experts

Mixtral: 8 experts stored, top-2 active. Collapse is a dense net in disguise.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A dense transformer uses **every** MLP on every token. A **mixture of experts (MoE)** keeps many MLPs (**experts**) and runs only a few of them per token. Capacity grows with the expert count; FLOPs per token stay closer to a small dense net.

1. What a mixture of experts (MoE) is

A dense transformer pays every feedforward block on every token. A **mixture of experts (MoE)** stores many copies of that MLP—the **experts**—and, for each token, runs only a few of them. A **router** (a linear map plus a softmax or a noisy top- k) picks which experts fire. Capacity, meaning stored parameters, grows with the expert count E . Compute per token stays closer to k dense MLPs, not E .

The name is literal. You still have attention. You still have a residual stream. You have replaced “one MLP” with “ E MLPs and a gate.” Mixtral’s row to memorize: **8 routed experts, top-2 active, no shared expert**. Switch Transformer used $k = 1$. DeepSeek-style 256-expert nets wait; this hour is stored versus active.

Sparse means most experts are idle for a given token. That is the point: you can store $8 \times$ the MLP weights and still pay about 2 experts of compute. If the router always picks the same two, you stored six idle copies and trained a dense net in disguise. Collapse is a failure of the method, not a different architecture.

2. Why we use it

Dense nets pay every MLP on every token. If you want more capacity, a dense recipe grows both the parameter file and the FLOPs per token. MoE tries to buy **width in parameters without width in compute**: more experts on disk, roughly k experts in the FLOP count.

That split matters for a project. A dense 7B is one weight file and a dense GEMM. An 8-expert Mixtral-style net is about eight MLP shards plus attention. You use an MoE if you need more capacity at a similar **token-FLOP** budget and you can host the weights. You use a dense 7B if you need simple deployment. Week 7.3 is how you shrink whatever you picked.

Serving is why “same active FLOPs” is not “same ops team.” Tokens in one sequence may go to different experts, so batching is harder. RAM still holds all experts even if only two run. A “small” Mixtral still has a large footprint.

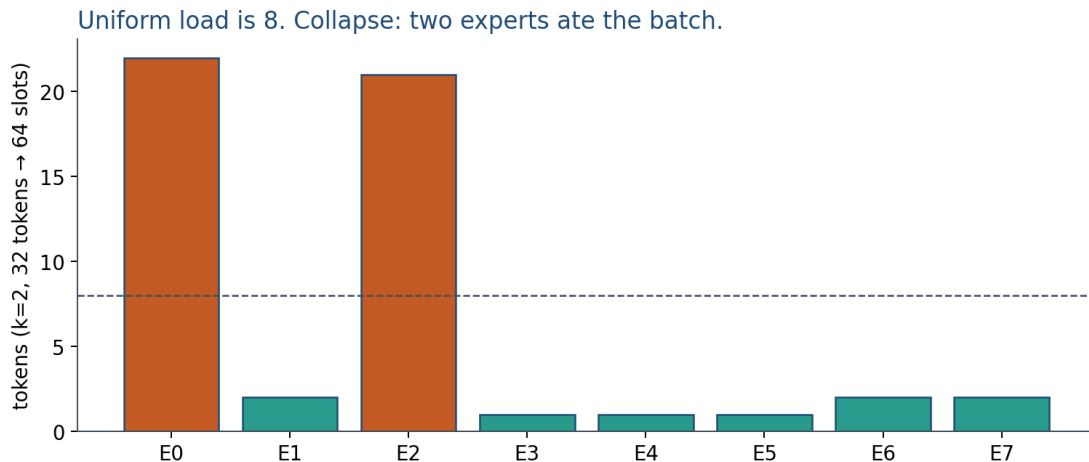
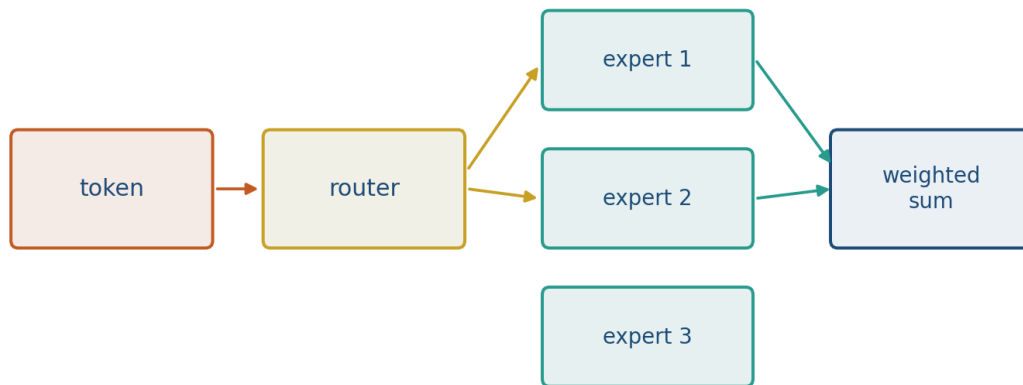
3. Architecture

Attention stays dense. The sparse part is the feedforward path, where most parameters already live (note **3.3**). Replace the block MLP with E experts $E_1, \dots, E_E$ and a **router**. For token x ,

$$y = \sum_{i \in \mathcal{T}(x)} g_i(x) E_i(x),$$

where $\mathcal{T}$ is the set of k selected experts (often $k = 1$ or 2) and g_i are the gate scores. After top-2, you typically renormalize the two g_i to sum to 1 so y is a weighted mix, not a raw sum of two MLPs.

Sparse MoE: most experts stay off for a given token



If each expert has P weights, you store about EP MLP weights plus a tiny router, but you **compute** only k experts. Active MLP FLOPs scale with k , not E . Mixtral-style: 8 stored, top-2 active, no shared expert. There is no extra transformer family here; there is a router sitting where the dense MLP used to sit.

4. How it works, step by step

Take one token and eight experts.

1. **Router logits.** A linear map produces a vector $z \in \mathbb{R}^E$. Example: $z = (3.0, 0.1, 2.5, 0, \dots)$.

2. **Select.** Softmax (or noisy top- k) then keep the k largest. Top-2 on that z picks experts 1 and 3.
3. **Run those experts** and mix with gate scores g_i . Renormalize the shortlist so the gates sum to 1.
4. **Load-balance.** If 100 tokens each pick $k = 2$, you have 200 expert-slots to fill. A uniform load is $200 / E$ tokens per expert. If expert 0 gets 150 of them, you trained a dense net in disguise and left seven experts cold. Switch-style load-balancing losses add a term that encourages the average gate to be $1 / E$. You do not need the formula in this course; you need to say **whether** you used one.
5. **Capacity.** A per-expert **capacity** c (tokens per batch) is the other knob. Over capacity $\Rightarrow$ those tokens skip the expert or fall back to a residual. Cartoon: 16 tokens, $k = 1$, $c = 2$. If the router sends 6 tokens to expert 0, four of them are **dropped**.

Routing is discrete. Gradients flow through the chosen experts and through the gate (straight-through or a softmax over a shortlist). Expert collapse and token dropping are the usual failure modes. If every token has the same z , $\mathcal{T}$ is always $\{1, 3\}$: you trained **two** MLPs and stored six idle copies.

At inference, experts must sit in RAM or be paged. Fine-tuning one expert with a frozen router and claiming “the MoE learned” is a report bug: say whether you trained the router.

5. Mathematical formulas

Sparse mix of selected experts:

$$y = \sum_{i \in \mathcal{T}(x)} g_i(x) E_i(x).$$

Stored MLP weights scale as EP ; active FLOPs scale with k . Per token you pay about k / E of “run every expert” compute, plus cheap routing. Parameter storage is still about $E \times$ the dense MLP (plus attention, which stayed dense).

Uniform-load cartoon: T tokens, top- k , E experts. Expected tokens per expert if routing is uniform:

$$\frac{Tk}{E}.$$

If expert 0 receives far more than that, and capacity is c , overflow is $\max(0, n_0 - c)$. Load-balancing tries to stop that before it starts.

For a typical block, each expert has $12d^2$ weights and attention has $4d^2$. For $E = 8$, most of the **stored** block weights sit in the experts. That fraction is the homework; the lecture claim is simpler: stored $\neq$ active.

6. Positive points and negative points

Positive.

- You can store many more MLP parameters than you run per token, so capacity grows without a matching FLOP tax.
- Mixtral-style $k = 2$ of 8 is a concrete stored-versus-active picture: 8 MLPs on disk, 2 MLPs in the FLOP count.
- Attention can stay dense; you only sparsify the path that already held most of the weights.

Negative.

- RAM (or paging) still holds all experts. A “small” Mixtral is not a small file.
- Batching is harder because tokens in one sequence may go to different experts. Same active FLOPs is not the same ops team.
- Router collapse trains a dense net in disguise and leaves experts cold.
- Token dropping at capacity is a silent quality hit unless you log it.
- Fine-tuning one expert with a frozen router is not “the mixture improved.”

When not to. Use a dense 7B if you need simple deployment. Do not pick MoE only because the marketing number sounds like a 56B dense model; Mixtral-style $8 \times 7B$ is not 56B dense at decode.

7. Teaching this note

About **35 minutes** at the board: draw one token, eight experts, top-2, then a collapse example (all mass on expert 2). Play the Deep Dive **internals** excerpt so the dense MLP is on screen, then say Mixtral routing is the board picture. Lab 7 is scaling-fit and quantization, not an MoE train.

End with stored vs active: 8 MLPs on disk, 2 MLPs in the FLOP count. If that sentence is not said out loud, Mixtral-style marketing will win.

8. Worked example

$E = 8$ experts, $k = 2$, each expert an MLP with the same FLOPs as a dense block MLP. Per token you pay $\approx 2 / 8 = 25\%$ of “run every expert” compute, plus cheap routing. Parameter storage is still $\approx 8 \times$ the dense MLP (plus attention, which stayed dense).

Router logits $z = (3.0, 0.1, 2.5, 0, \dots)$. Softmax then top-2 picks experts 1 and 3. If every token has the same z , $\mathcal{T}$ is always $\{1, 3\}$: you trained **two** MLPs and stored six idle copies.

Capacity, cartoon: 16 tokens, $k = 1$, expert capacity $c = 2$ tokens. If the router sends 6 tokens to expert 0, four of them are **dropped** (or overflowed to a residual). Load-balancing tries to stop that before it starts.

Gate scores: after top-2, renormalize the two g_i to sum to 1 so y is a weighted mix, not a raw sum of two MLPs. Mixtral-style $k = 2$ is that mix.

9. Where students get stuck

- Counting **stored** parameters as **active** FLOPs. Mixtral-style $8 \times 7\text{B}$ is not 56B dense at decode.
 - Fine-tuning one expert with a frozen router and claiming “the MoE learned.”
 - Forgetting RAM: all experts must be loaded even if only two run.
-

10. Video

[Karpathy — Deep Dive into LLMs like ChatGPT](#). Assigned excerpt: **20:11–26:01** (neural-net internals: the dense MLP). The Deep Dive does not isolate Mixtral; **routing is the board hour**. Pause on the feedforward block and replace it, in chalk, with a router and E copies.

11. Practice

1. If $k = 1$ and the router always picks expert 2, what did you just train?
2. Why can an MoE with 8 experts have roughly the compute of 2 dense MLPs per token, not 8?
3. Name one reason Mixtral-style serving is harder than a dense model with the same active FLOPs.
4. $E = 8$, $k = 2$, 32 tokens, uniform routing. Expected tokens per expert? If expert 0 receives 20 tokens and capacity is 6, how many overflow?
5. Each expert has $12d^2$ weights (a typical MLP). Attention is $4d^2$. For $d = 4096$, $E = 8$, what fraction of **stored** block weights sit in the experts? (Ignore biases and the router.)